TuitionHub BD

System Design Document

Version 1.0

July 2026

A Nationwide Tuition Marketplace for Bangladesh

Connecting University Students with Families Seeking Tutors

Prepared for internal architecture review — Implementation follows TDD methodology

Tech Stack: Next.js 15 + ASP.NET Core 9 + PostgreSQL 16 + FastAPI (AI/ML) + Docker

Contents

Chapter 1

Executive Summary

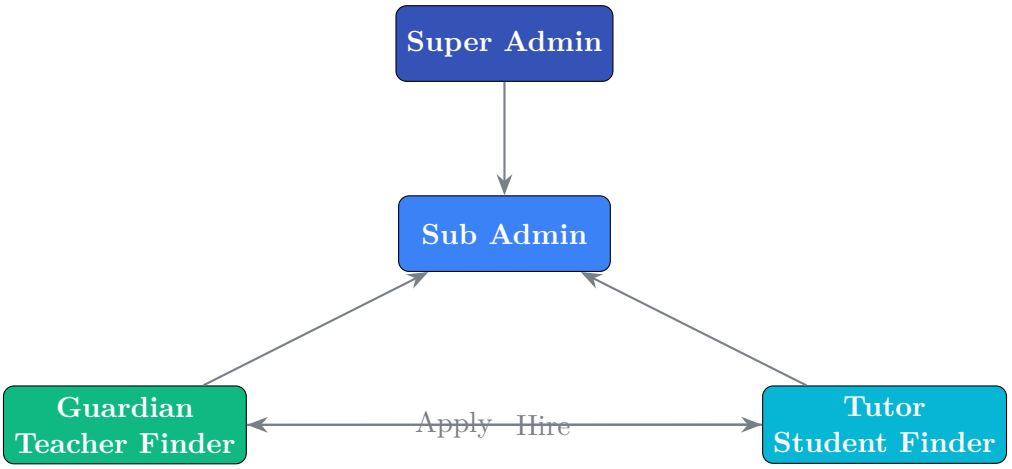
1.1 Purpose

TuitionHub BD is a multi-sided marketplace platform that connects **university students seeking tuition jobs (Tutors)** with **guardians or students looking for qualified private tutors (Guardians)** across Bangladesh. The platform is managed by a **Super Admin** and supported by **Sub Admins**.

1.2 Core Objectives

- Create a trusted, transparent marketplace for private tuition in Bangladesh
- Implement a fair, multi-factor tutor matching system driven by rules and ML, not LLMs
- Provide role-specific dashboards for all four user types
- Enable secure communication, verified profiles, and reputation-based trust scoring
- Build for scale — millions of users across all divisions of Bangladesh

1.3 Four Primary Roles



Chapter 2

System Architecture

2.1 High-Level Architecture Diagram

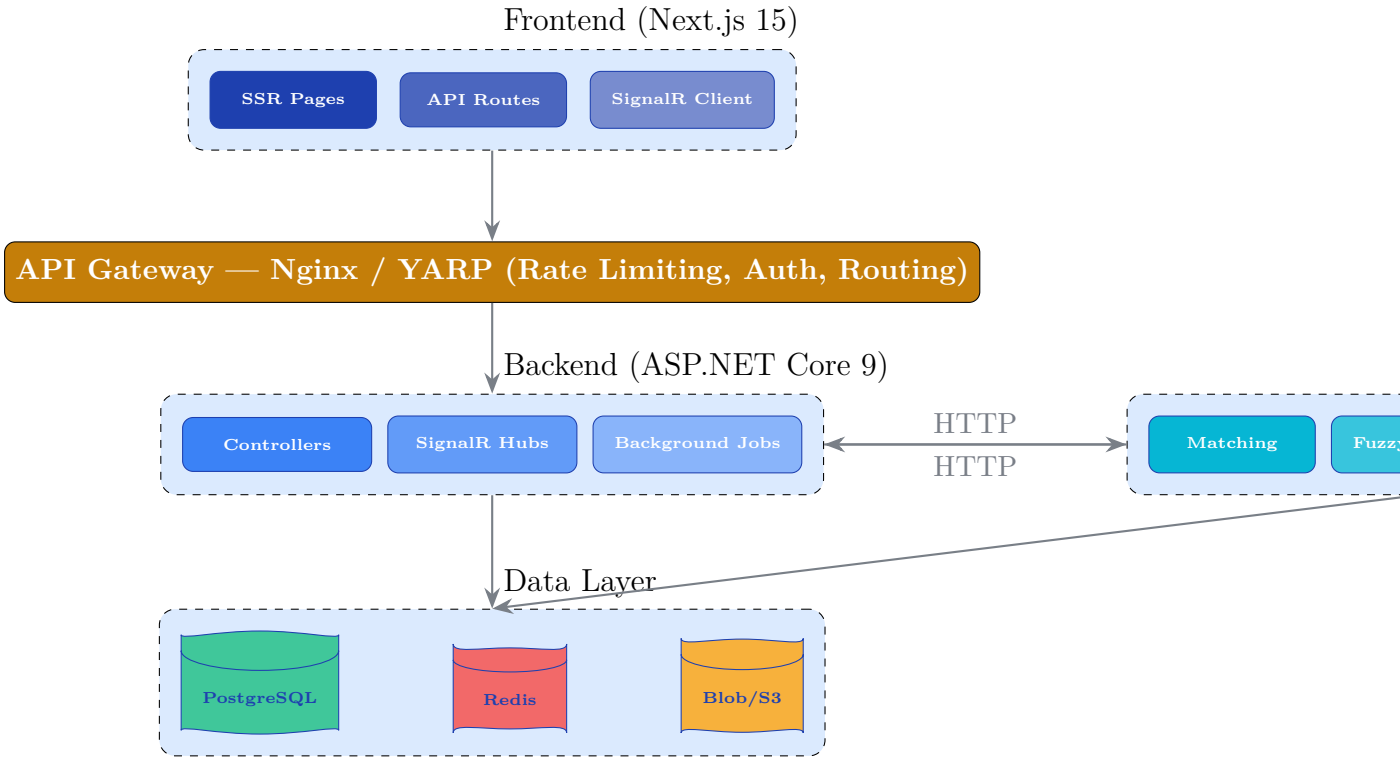


Figure 2.1: **High-Level System Architecture — Three-Service Topology**

2.2 Technology Stack

Layer	Technology	Rationale
Frontend	Next.js 15 (App Router) + React 19	SSR, SEO, file-based routi
Styling	Tailwind CSS v4 + Motion (Framer Motion successor)	Utility-first, dark mode, ar
Components	shadcn/ui + Magic UI (extracted)	150+ animated component
State	Zustand + TanStack Query	Client state + server cache

Layer	Technology	Rationale
Forms	React Hook Form + Zod	Performant forms with sch
Backend	ASP.NET Core 9 Web API	High-performance, strongly
ORM	Entity Framework Core 9	Mature, LINQ provider, m
Auth	JWT Access + Refresh Token (HTTP-only cookies)	Stateless, secure, refresh ro
Real-Time	SignalR	Bidirectional, WebSocket v
Cache	Redis 7	Low-latency, session store,
Database	PostgreSQL 16 + pg_trgm, CIText, UUIDv7	JSONB, full-text search, tr
AI/ML Service	FastAPI + scikit-learn	Async, auto-docs, simple r
Container	Docker + Docker Compose	Reproducible environments
CI/CD	GitHub Actions	Tight GitHub integration,

Chapter 3

Role-Based Access Control

3.1 RBAC Architecture

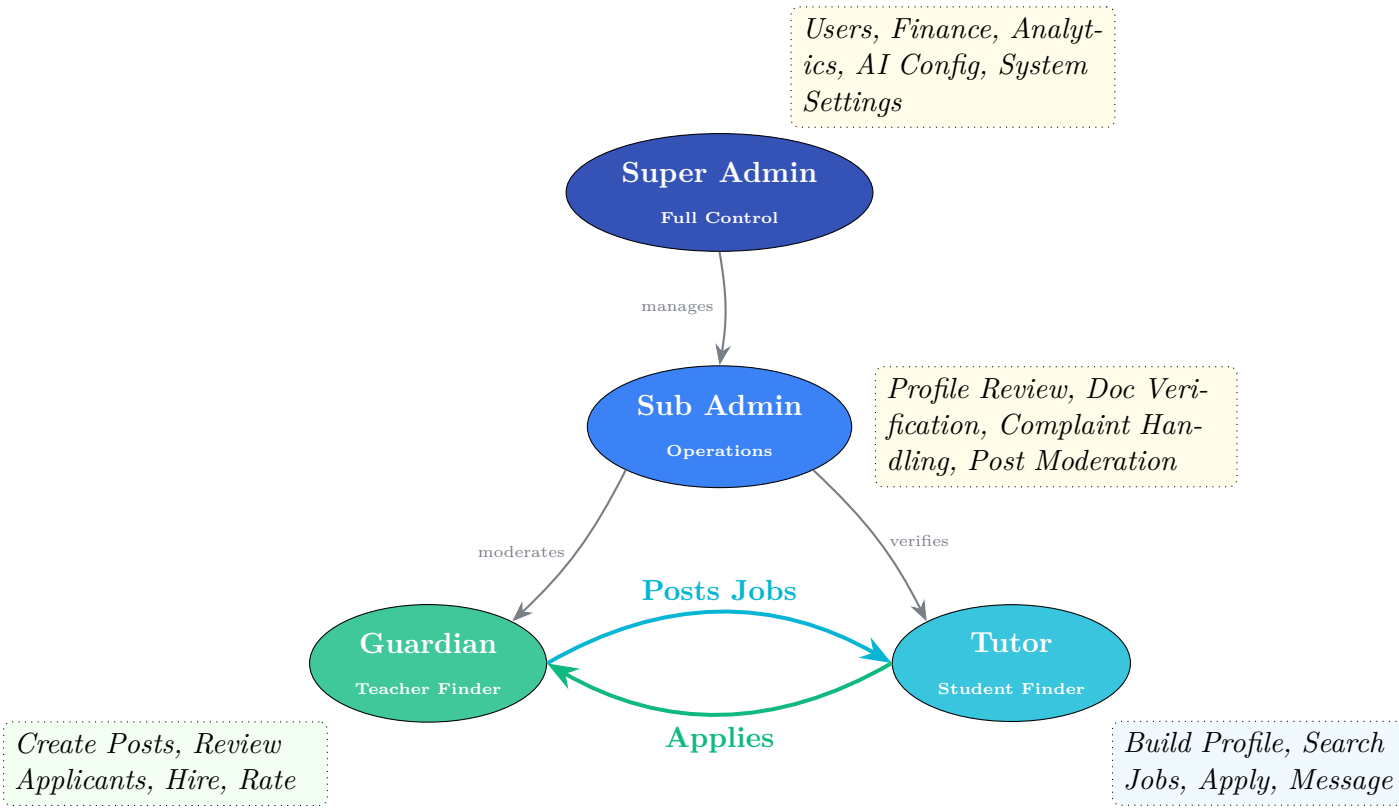


Figure 3.1: **Role Hierarchy & Permission Boundaries**

3.2 Permissions Matrix

Capability	Super Admin	Sub Admin	Guardian	Tutor
Manage Users	✓	✓	—	—
Verify Tutors/Documents	✓	✓	—	—

Capability	Super Admin	Sub Admin	Guardian	Tutor
View Financial Reports	✓	—	—	—
Configure Commission	✓	—	—	—
Access Audit Logs	✓	✓*	—	—
Handle Complaints	✓	✓	File only	File only
Approve Tuition Posts	✓	✓	—	—
Create Tuition Posts	—	—	✓	—
Browse & Apply Jobs	—	—	—	✓
Hire/Reject Tutors	—	—	✓	—
Rate & Review	—	—	✓	—
Live Chat	—	—	✓	✓
View AI Insights	✓	✓	Dashboard	Dashboard
Manage System Config	✓	—	—	—

* Sub Admin: limited to operations-related logs only

Chapter 4

Database Schema

4.1 Entity Relationship Diagram

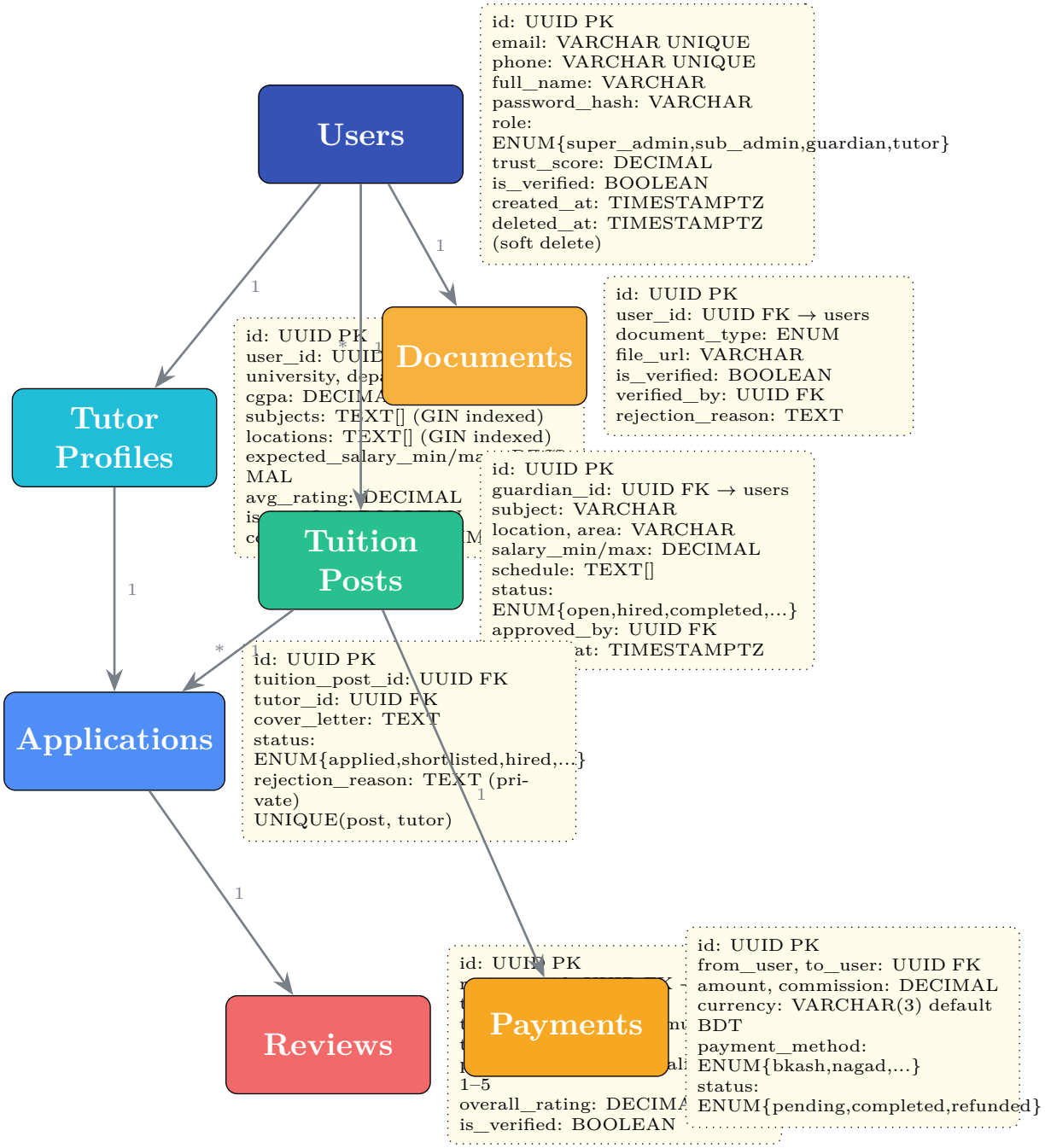


Figure 4.1: **Core Entity Relationship Diagram (Simplified)**

4.2 Complete Schema Listing

The full schema includes 13 tables:

Table	Key Indexes	Purpose
users	role, phone, trust_score DESC	Platform users, authentication, role assignment
refresh_tokens	(user_id), expires_at WHERE revoked	Secure JWT refresh token rotation
otp_codes	(user_id, type)	Email/phone verification, password reset
tutor_profiles	is_verified, avg_rating DESC, GIN(subjects, locations)	Extended tutor profile, search, matching
documents	user_id, is_verified	Verification document storage
tuition_posts	guardian_id, status(open), subject, location, salary	Job listings with full-text search
applications	(post_id, tutor_id), status	Job application workflow
reviews	tutor_id, overall_rating DESC	Multi-factor tutor rating system
guardian_verifications	user_id, verification_level, is_verified	Guardian NID/phone verification
payment_transactions	from_user, to_user, status	Revenue, commission, payouts
conversations / messages	(conversation_id, created_at), unread filter	Real-time chat
notifications	(user_id, is_read, created_at DESC)	In-app + push notifications
audit_logs	user_id, action, (entity_type, entity_id)	Compliance, monitoring
guardian_verifications	user_id, verification_level, is_verified	Guardian NID/phone/email verification
system_config	key UNIQUE	Dynamic configuration (commission, weights)

4.3 Key Indexing Strategy

Query Pattern	Index Type	Details
Browse open tuition posts	Partial B-tree	WHERE status = 'open' ORDER BY created_at DESC
Fuzzy search subjects/locations	GIN trigram	pg_trgm extension, similarity() function
Tutor recommendations	Multi-column	avg_rating DESC, trust_score DESC, total_hires DESC
Search tutors by subject/location	GIN array	teaching_subjects[], preferred_locations[]
Guardian's active posts	Composite B-tree	(guardian_id, status)
Unread notifications	Partial B-tree	(user_id, is_read) WHERE is_read = FALSE
Active complaints	Partial B-tree	status WHERE status IN ('open', 'investigating')
Payment history	B-tree	from_user_id, to_user_id, created_at DESC
Chat messages	Composite B-tree	(conversation_id, created_at)

Chapter 5

Backend Architecture

5.1 Clean Architecture Layers

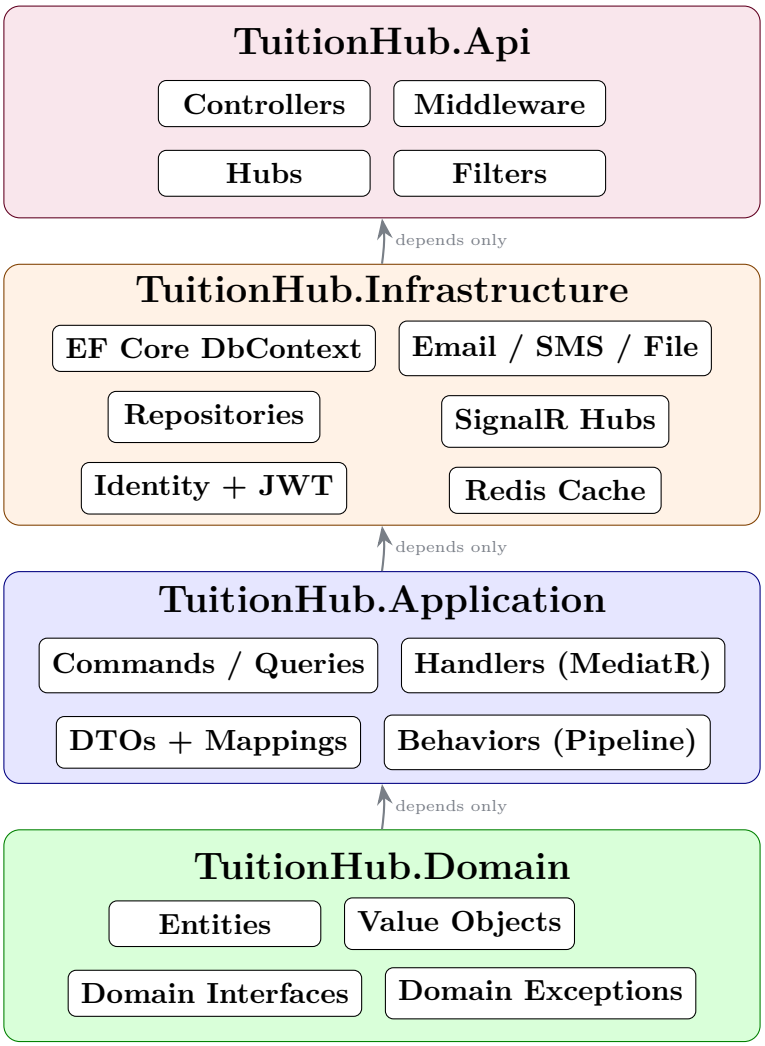


Figure 5.1: **Clean Architecture — Dependency Inversion Principle**

5.2 CQRS with MediatR

Each use case is represented as a `Command` (write) or `Query` (read), handled through MediatR with pipeline behaviors for cross-cutting concerns:

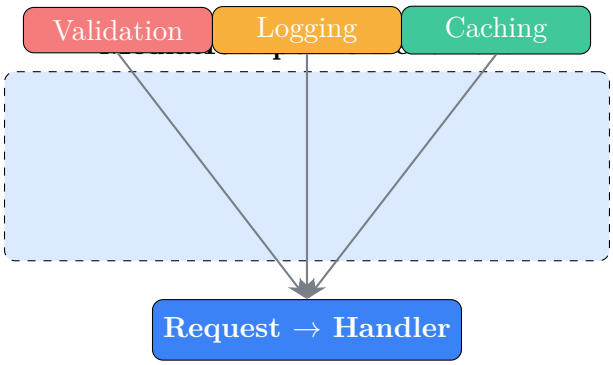


Figure 5.2: **MediatR Pipeline with Cross-Cutting Behaviors**

5.3 JWT Authentication Flow

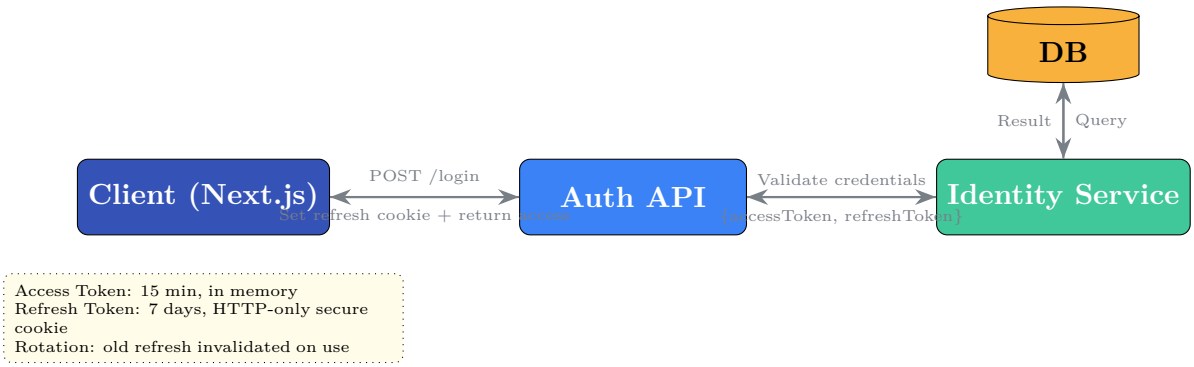


Figure 5.3: **JWT Access + Refresh Token Architecture**

Chapter 6

Frontend Architecture

6.1 Component Architecture

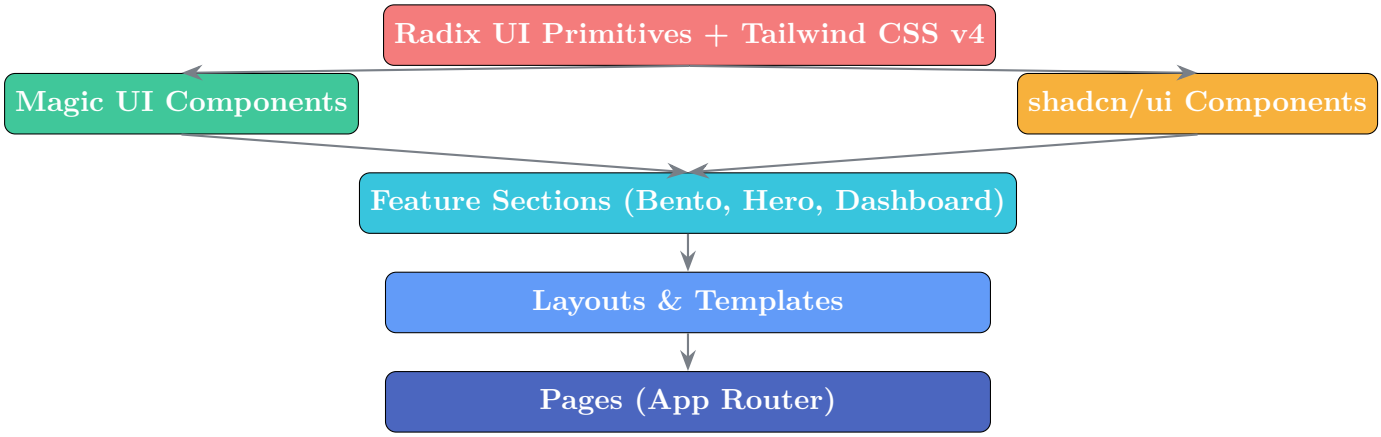


Figure 6.1: **Frontend Component Hierarchy**

6.2 Route Structure

Group	Routes	Access	Content
Marketing	/, /about, /pricing, /contact	Public	Landing page (Magic UI hero, bento, marquee), SEO-optimized
Auth	/login, /register, /forgot-password, /verify-otp	Public	Forms with Zod validation, OTP input, password strength
Jobs	/jobs, /jobs/[id]	All	Tuition listing with fuzzy search, filters, pagination
Tutors	/tutors, /tutors/[id]	All	Tutor search with matching score badges, profiles
Guardian	/dashboard/guardian/*	Guardian only	Posts CRUD, applicants, hires, reviews, chat

Group	Routes	Access	Content
Tutor	/dashboard/tutor/*	Tutor only	Profile, applications, job search, reviews, chat
Admin	/dashboard/admin/*	Super/Sub Admin	User mgmt, verifications, finance, analytics, complaints

Chapter 7

Matching Engine & Search

7.1 Tutor Matching Algorithm

The matching engine is a **pure rule-based scorer** with configurable weights stored in the database (system_config table). No LLM, no neural networks.

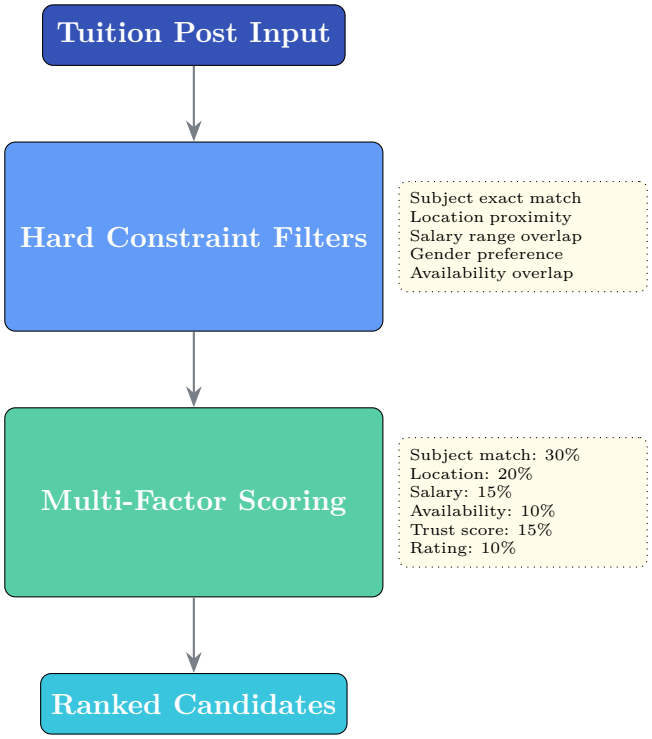


Figure 7.1: **Matching Pipeline — Constraints → Scoring → Ranking**

7.1.1 Scoring Formula

```
Score = subject_match * 0.30 +
        location_match * 0.20 +
        salary_overlap * 0.15 +
        availability_overlap * 0.10 +
        (trust_score / 5.0) * 0.15 +
        (avg_rating / 5.0) * 0.10
```

All weights are configurable by Super Admin via the system_config table, allowing dynamic tuning without code deployment.

7.2 Fuzzy Search Implementation

PostgreSQL `pg_trgm` extension provides trigram-based fuzzy matching:

- **Trigram Index**: All subject names, locations, and post titles are decomposed into 3-character sequences
- **Similarity Operator** (`%`): Returns results where trigram similarity exceeds threshold
- **Distance Function** (`<->`): Used for sorting by closest match
- Works for both **Bangla and English** text

Listing 7.1: Fuzzy Search Query Example

```sql
-- Enable extension
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Trigram index on searchable fields
CREATE INDEX idx_posts_title_trgm ON tuition_posts
    USING GIN (title gin_trgm_ops);
CREATE INDEX idx_posts_subject_trgm ON tuition_posts
    USING GIN (subject gin_trgm_ops);
CREATE INDEX idx_posts_location_trgm ON tuition_posts
    USING GIN (location gin_trgm_ops);

-- Fuzzy search with ranking
SELECT id, title, subject, location,
       similarity(title, 'math␣ttor') AS sim_score
FROM tuition_posts
WHERE status = 'open'
  AND (title % 'math␣ttor' OR subject % 'math')
  AND location % 'dhanmondi'
ORDER BY sim_score DESC
LIMIT 20;
```

7.3 Salary Prediction

A **linear regression model** (scikit-learn) trained on historical completed hire data:

Listing 7.2: Salary Prediction Model

```python
import sklearn.linear_model
import numpy as np

# Features: subject_encoded, location_encoded, class_grade,
#           tutor_experience, tutor_cgpa, teaching_medium
model = sklearn.linear_model.LinearRegression()
model.fit(X_train, y_train)  # trained on completed hires
```

```

def predict_salary(features: dict) -> dict:
    X = np.array([[features['subject_enc'], features['location_enc'],
                   features['grade_enc'], features['experience'],
                   features['cgpa'], features['medium_enc']]])
    predicted = model.predict(X)[0]
    return {
        'predicted_min': round(predicted * 0.8),
        'predicted_max': round(predicted * 1.2),
        'recommended': round(predicted),
        'confidence': model.score(X_test, y_test)
    }
```

7.4 Fraud Detection

Hybrid approach: **rule-based scoring** + **Isolation Forest** anomaly detection:

Listing 7.3: Fraud Risk Score Calculation

```
def fraud_risk_score(profile: dict, behavior: dict) -> dict:
    risk = 0

    # Rule-based checks
    if profile.get('age', 0) < 18: risk += 25
    if profile.get('university') == '': risk += 10
    if not profile.get('documents_submitted'): risk += 20
    if behavior.get('login_frequency', 0) > 100: risk += 15
    if behavior.get('device_count', 0) > 3: risk += 10

    # Anomaly detection (Isolation Forest)
    features = np.array([[...features...]])
    anomaly_score = isolation_forest.decision_function(features)
    if anomaly_score < -0.5: risk += 20  # outlier detected

    # Classification
    if risk < 30:       level = 'low'
    elif risk < 60:     level = 'medium'
    else:               level = 'high'

    return {'risk_score': risk, 'risk_level': level}
```

7.5 Trust Score Calculation

```
trust_score = (
    (avg_rating / 5.0) * 0.30 +          # rating contribution
    completion_rate * 0.20 +              # completed / hired
    normalized_response_time * 0.10 +     # faster = higher score
    log_scale(total_hires) * 0.10 +       # experience
    profile_completeness * 0.10 +         # % of fields filled
    (1.0 if verified else 0.3) * 0.10 +   # verification bonus
    (1.0 - complaint_penalty) * 0.10      # complaint deduction
) * 5.0  # scale to 5.0
```

Chapter 8

API Design

8.1 API Response Envelope

Listing 8.1: Standard API Response (JSON)

```
{
    "success": true,
    "data": { ... },
    "message": "Operation completed successfully",
    "errors": [],
    "meta": {
        "page": 1,
        "pageSize": 20,
        "totalCount": 150,
        "totalPages": 8
    }
}
```

8.2 API Endpoint Map

Method	Endpoint	Description
Authentication		
POST	/api/v1/auth/register	Register guardian or tutor
POST	/api/v1/auth/login	Login → JWT access + refresh token
POST	/api/v1/auth/refresh	Rotate refresh token
POST	/api/v1/auth/verify-otp	Verify email/phone via OTP
POST	/api/v1/auth/forgot-password	Send password reset OTP
POST	/api/v1/auth/reset-password	Reset password with OTP
Users		
GET	/api/v1/users/me	Current user profile
PUT	/api/v1/users/me	Update profile
POST	/api/v1/users/me/documents	Upload verification documents
GET	/api/v1/users/{id}	Public user profile

Method	Endpoint	Description
Tutors		
GET	/api/v1/tutors	Search/filter tutors with fuzzy search
GET	/api/v1/tutors/{id}	Tutor public profile with rating
GET	/api/v1/tutors/{id}/reviews	Paginated tutor reviews
POST	/api/v1/tutors/recommendations	AI-matched tutor recommendations
Tuition Posts		
GET	/api/v1/tuition-posts	Browse with fuzzy search, filters, pagination
POST	/api/v1/tuition-posts	Guardian creates tuition post
PUT	/api/v1/tuition-posts/{id}	Update post
PATCH	/api/v1/tuition-posts/{id}/status	Change post status
DELETE	/api/v1/tuition-posts/{id}	Soft-delete post
Applications		
POST	/api/v1/applications	Tutor applies to tuition post
GET	/api/v1/applications	List applications (filter by user/post)
PATCH	/api/v1/applications/{id}/shortlist	Guardian shortlists
PATCH	/api/v1/applications/{id}/hire	Guardian hires
PATCH	/api/v1/applications/{id}/reject	Guardian rejects with private feedback
Reviews		
POST	/api/v1/reviews	Create multi-factor review
GET	/api/v1/reviews	List reviews (filter by tutor)
PATCH	/api/v1/reviews/{id}/flag	Admin flags suspicious review
Payments		
POST	/api/v1/payments/commission	Calculate commission
GET	/api/v1/payments/transactions	Payment history
GET	/api/v1/payments/reports	Admin revenue reports
Chat		
GET	/api/v1/chat/conversations	List conversations
POST	/api/v1/chat/conversations	Create conversation
GET	/api/v1/chat/conversations/{id}/messages	Message history
POST	/api/v1/chat/conversations/{id}/messages	Send message
Admin		
GET	/api/v1/admin/dashboard	Aggregated statistics
GET	/api/v1/admin/users	User management list
PATCH	/api/v1/admin/users/{id}/verify	Verify tutor
DELETE	/api/v1/admin/users/{id}	Suspend/ban user
GET	/api/v1/admin/audit-logs	Compliance audit
GET	/api/v1/admin/reports/{type}	Business reports

Chapter 9

Business Model & Monetization Strategy

9.1 Proposed Model Overview

Revenue Stream	Who Pays	When
Commission (30% of 1st month salary)	Tutor	Upon successful hire
Registration Fee (BDT 500, one-time)	Tutor	After free launch period
Premium Verified Badge (BDT 5,000)	Tutor	Growth phase (eligibility-gated)
Freemium Guardian Features	Guardian	Scale phase
Advertising & Sponsorships	Advertisers	Growth phase onward

9.2 Revenue Stream Analysis

9.2.1 Core: 30% Commission on First Month's Salary

Target: Tutors who get hired through the platform.
Mechanism: One-time deduction from the first month's tuition payment. No recurring fees.
Rationale: Aligns platform revenue with value delivered — tutor only pays when they actually get hired.

Dimension	Assessment
Advantages	Low upfront barrier, highly scalable, perceived as fair (pay-per-success)
Disadvantages	Revenue is delayed (only realized after hire), high rate (30%) may incentivize platform bypass

Dimension	Assessment
Risk	Tutors and guardians may attempt to connect once and transact offline
Mitigation	In-app messaging enforced until hire confirmation. Contact info released only after commission recorded.
User Acceptance	Moderate — tutors will compare with free Facebook groups, so value proposition must be clear
Implementation	Phase 3 (MVP) — payment integration with bKash/Nagad

9.2.2 Registration Fee: BDT 500 (One-Time)

Target: New tutors registering after the free launch period.
Mechanism: One-time payment to activate tutor account.
Purpose: Filters unserious applicants, generates baseline revenue, signals commitment.

Dimension	Assessment
Advantages	Filters low-quality signups, generates early revenue, barrier relative to earning potential
Disadvantages	May slow supply-side growth, creates friction for genuine but cash-strapped students
Risk	Competitors offering free signups may capture market share
Mitigation	Introduce only after critical mass of tuition posts exists (demand-pull justifies fee)
User Acceptance	Low to moderate initially — requires strong value proposition
Implementation	Phase 4 (Growth) — not during MVP

9.2.3 Premium Verified Badge: BDT 5,000

Target: Tutors seeking higher visibility and credibility.
Mechanism: Eligibility-gated badge — the platform evaluates the tutor first, then allows purchase only if criteria are met. This is **not** pay-to-win.

Eligibility Criteria:

- Identity verified (NID, student ID, or passport)
- Academic documents verified (transcript, certificate)
- Minimum trust score ≥ 4.0
- No unresolved complaints
- Minimum 3 completed tuition jobs
- Profile completeness $\geq 90\%$

Benefits:

- Priority placement in search results (boosted weight in matching engine)
- Featured on homepage tutor carousel
- Premium "Verified" badge on profile
- Higher visibility in recommendation engine
- Priority customer support

Dimension	Assessment
Advantages	High-margin, builds trust ecosystem, incentivizes profile quality
Disadvantages	Complex verification workflow, manual review overhead initially
Risk	Perceived as elitist or unfair if criteria are unclear
Mitigation	Transparent criteria published; Admin review + AI-assisted verification
User Acceptance	High — tutors want trust signals to stand out
Implementation	Phase 5 (Growth) — after stable marketplace exists

9.2.4 Freemium Guardian Features

Target: Guardians posting tuition jobs.
Free tier: Post tuition jobs, review applicants, hire tutors, rate/review.
Premium features (future):

- Featured/boosted tuition post (higher visibility to tutors)
- Urgent hiring package (express matching within 24 hours)
- Background check report on shortlisted tutors
- Priority customer support

9.2.5 Secondary Revenue Streams (Growth & Scale)

Stream	Phase	Est. Complexity	Notes
Google AdSense / Display Ads	Growth	Low	Non-intrusive banner ads on public pages
Sponsored Tutor Profiles	Growth	Medium	Premium placement in search results

Stream	Phase	Est. Complexity	Notes
Sponsored Tuition Posts	Growth	Medium	Guardian pays to boost post visibility
Affiliate Program (tutor referral)	Growth	Medium	Referral bonus for bringing new tutors
Recruitment Partnerships	Scale	High	API/portal for coaching centers to bulk-hire
Digital Achievement Badges	Growth	Low	Certificates for completed jobs,
Online Course Promotions	Scale	Medium	Partner with 10 Minute School, Shikho
Background Verification Service	Scale	High	Third-party verification for guardians too

9.3 Phased Monetization Roadmap

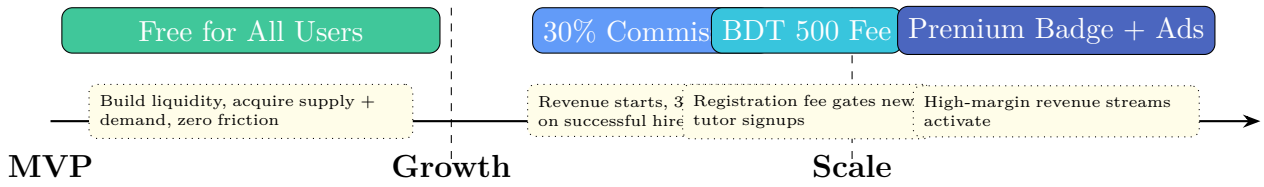


Figure 9.1: **Monetization Phasing**

Chapter 10

Go-to-Market Strategy

10.1 The Chicken-and-Egg Problem

Every marketplace faces this: **Tutors won't join without tuition posts; guardians won't post without tutors.** The solution is a **supply-first (tutor-first) acquisition strategy**, executed in phases.

10.2 Phase 1: Supply-First (Dhaka Pilot — Months 0–2)

Strategy: Recruit a critical mass of high-quality tutors before marketing to guardians.

Tactic	Execution
Campus Ambassador Program	Recruit 1 ambassador per major university (DU, BUET, JU, NSU, BRAC, etc.). Ambassadors get BDT 200 per verified tutor referral.
Free Profile Setup Drive	First 1,000 tutors get lifetime free profiles (grandfathered). Creates urgency.
University Partnerships	Partner with university career centers to promote the platform as a reliable tuition-finding tool.
Social Proof	Highlight top tutors from prestigious universities to attract both tutors and guardians.
Dhaka-Only Focus	All marketing and operations confined to Dhaka for measurable, concentrated growth.
Target:	**2,000+ verified tutors in Dhaka** before opening to guardians.

10.3 Phase 2: Demand Activation (Months 2–4)

Strategy: Turn on guardian acquisition with a built-in tutor pool.

Tactic	Execution
Guardian Free Posting	Tuition posts are completely free. Zero friction to create a post.
Facebook Group Infiltration	Target the largest tuition-related Facebook groups ("Tuition in Dhaka", "Tutor Wanted Bangladesh") with targeted posts and ads.
Referral Incentive	Guardians who post get BDT 200 credit when their post leads to a successful hire.
Search Engine Optimization	Optimize landing pages for "Dhaka tutor", "private tutor in Bangladesh" keywords.
Community Building	Create Facebook group and WhatsApp broadcast channels for guardians and tutors separately.
Target:	**500+ active tuition posts and 50+ successful hires** in Dhaka.

10.4 Phase 3: Network Effects (Months 4–6)

Strategy: Leverage successful hires to drive organic growth.

- Successful hires → guardians tell other guardians (word-of-mouth)
- Tutors earn money → they recruit their friends (organic tutor supply)
- Reviews and ratings build trust → lowers barrier for new users
- Introduce the 30% commission once the marketplace is liquid
- Expand to Chattogram, then other divisional cities

10.5 Guardian Verification

10.5.1 Why It Matters

Fake tuition posts, harassment, and unreliable clients are common in Bangladesh. Tutors need assurance that the guardians they engage with are legitimate.

10.5.2 Verification Levels

Level	Requirement	Benefit
Basic	Phone OTP verification	Required for all guardians. Phone is verified via SMS OTP at registration. Protects against anonymous/spam posts.

Level	Requirement	Benefit
Verified	Phone + NID upload (optional)	Displays Verified Guardian badge. Increases tutor confidence. Higher trust score weight.
Institutional	School/college email domain	For institutions posting bulk tuition needs. Highest trust tier.

10.6 Bangla Language Search Strategy

PostgreSQL `pg_trgm` with additional preprocessing for production Bangla search:

Listing 10.1: Bangla-Aware Fuzzy Search

```
-- Unicode normalization (NFKC) handles different
-- representations of the same Bangla character
CREATE INDEX idx_posts_bangla_trgm ON tuition_posts
    USING GIN (normalize(title, NFKC) gin_trgm_ops);
CREATE INDEX idx_tutors_subjects_bangla ON tutor_profiles
    USING GIN (normalize(teaching_subjects::text, NFKC) gin_trgm_ops);

-- Phonetic search for common Bangla variations:
--    / ganit / math → all match "mathematics"
--    / dhaka → both match
SELECT * FROM tuition_posts
WHERE normalize(location, NFKC) % normalize('  ', NFKC)
   OR location % 'dhaka'
ORDER BY
    CASE WHEN normalize(location, NFKC) % normalize('  ', NFKC)
         THEN similarity(normalize(location, NFKC), normalize('  ', NFKC))
         ELSE similarity(location, 'dhaka')
    END DESC;
```

Additional Bangla considerations:

- Unicode normalization (NFKC) to handle equivalent character sequences
- Common Romanization mappings: `sh=` / / , `dh=` / , `ng=`
- Keyboard variation handling: Avro Phonetic vs. Bijoy layout differences
- Application-layer normalization: normalize Bangla text to a canonical form before indexing and querying
- Dictionary of common spelling variants for high-frequency search terms

10.7 Mobile-First: Progressive Web App (PWA)

From day one, the frontend is built as a PWA:

Feature	Implementation
Service Worker	Cache static assets and API responses for offline access
Web Manifest	Full-screen install prompt on Android/iOS
Push Notifications	Delivery via Service Worker + Firebase Cloud Messaging
Offline Support	Cached tutor profiles, tuition posts, and chat history
Responsive Design	Mobile-first Tailwind CSS breakpoints for all components

Why PWA first, not native:

- 80%+ of Bangladeshi internet users are mobile-first
- PWA covers Android instantly (Play Store install not required)
- No app store approval delays, no 30% Apple tax on commissions
- Native apps can be developed in Scale phase after PMF is proven

Chapter 11

Infrastructure & Deployment

11.1 Deployment Architecture

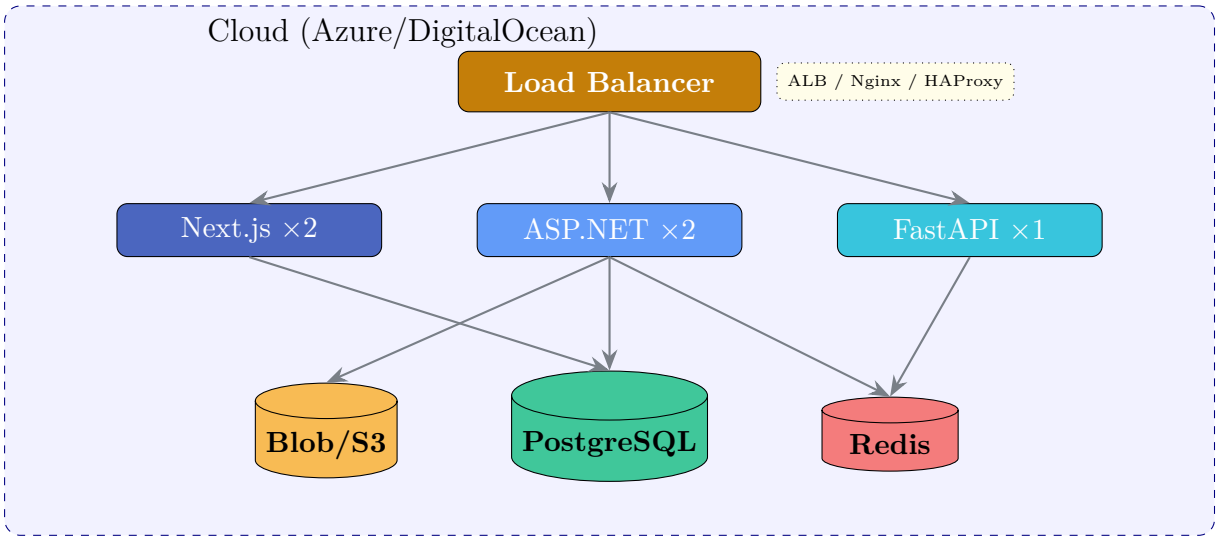


Figure 11.1: **Production Deployment Topology**

11.2 CI/CD Pipeline (GitHub Actions)

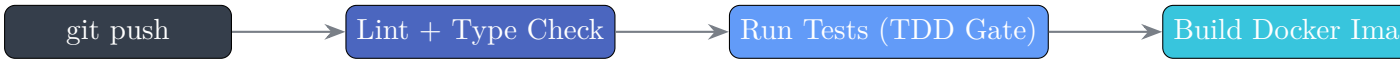


Figure 11.2: **CI/CD Pipeline Stages**

11.3 Container Strategy (Docker)

Listing 11.1: docker-compose.yml (Development)

```
services:
  postgres:
    image: postgres:16-alpine
    volumes: [pgdata:/var/lib/postgresql/data]
```

```
    environment:
      POSTGRES_DB: tuitionhub
      POSTGRES_USER: tuitionhub
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    ports: ["5432:5432"]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  backend:
    build: ../backend
    depends_on: [postgres, redis]
    ports: ["5000:8080"]

  ai-service:
    build: ../ai-service
    depends_on: [redis]
    ports: ["8000:8000"]

  frontend:
    build: ../frontend
    depends_on: [backend]
    ports: ["3000:3000"]
```

11.4 Estimated Resources (Initial Launch)

Service	vCPU	RAM	Storage	Instances
Next.js Frontend	2	4 GB	—	2
ASP.NET Core API	4	8 GB	—	2
FastAPI AI Service	2	4 GB	—	1
PostgreSQL	4	16 GB	100 GB SSD	1 + 1 replica
Redis	2	4 GB	—	1
Monthly (est.)				**$300–500**

Chapter 12

Development Roadmap

12.1 Overview

The roadmap is structured to solve the chicken-and-egg problem first, validate the business model early (payment integration in Phase 3), and progressively introduce monetization as marketplace liquidity grows.

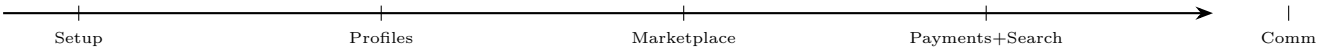


Figure 12.1: **8-Phase Build Timeline (16 Weeks)**

12.2 Build Phases

Ph	Weeks	Core Focus	Deliverables
1	1–2	Foundation	**Git + GitHub setup.** Docker (PostgreSQL + Redis). Clean Architecture scaffolding + Auth (TDD). JWT + Refresh token rotation. Login/Register UI (mobile-first responsive). Service Worker + PWA manifest scaffolding.
2	3–4	Profiles+Verif	User profiles. Tutor profile with academic/accreditation. Document upload. **Guardian verification**: phone OTP + optional NID. Admin verification flow. Profile UI with document preview.
3	5–6	Marketplace+Pay	Tuition posts CRUD + Apply/Shortlist/Hire + Fuzzy search (English + Bangla). **bKash/Nagad payment integration**: commission calculation, transaction recording, payment gateway SDK. Job browsing UI.

Ph	Weeks	Core Focus	Deliverables
4	7–8	AI Matching	Rule-based matching engine (configurable weights from DB). pg_trgm fuzzy search optimization. Salary prediction (LinearRegression). Fraud detection (rule-based + Isolation Forest). Trust score engine. Recommendation UI.
5	9–10	Communication	SignalR chat (real-time, WebSocket with fallback). Multi-factor reviews (6 dimensions). Complaint workflow. Notification system (in-app + push via Service Worker). Chat UI + review form.
6	11–12	Admin Panel	Super/Sub Admin dashboards. User management. Document verification queue. Financial reports. Audit log viewer. Admin panel UI with role-gated views.
7	13–14	Monetization	**Tutor registration fee (BDT 500)** gate. **Premium Verified Badge (BDT 5,000)** with eligibility workflow. Featured posts. Ad placements. Revenue dashboard. Monetization UI + payment collection.
8	15–16	Production	Security audit (OWASP Top 10). Load testing (k6). E2E tests (Playwright). Bangla search QA. PWA install test on Android/iOS. Staging deploy. Documentation. Production launch.

12.3 Go-to-Market Timeline

Period	Product Stage	Business Stage	Key Metrics
Month 1–2	MVP (Phases 1–2)	Supply acquisition	**2,000+ verified tutors in Dhaka**
Month 3–4	Launch (Phases 3–4)	Demand activation	**500+ tuition posts, 50+ hires**
Month 5–6	Growth (Phases 5–6)	Network effects	30% commission active, organic growth
Month 7–8	Scale (Phases 7–8)	Monetization	BDT 500 fee, Premium Badge, expansion

Period	Product Stage	Business Stage	Key Metrics
Month 9–12	Expansion	National rollout	Chattogram → Rajshahi → Khulna → Sylhet

12.4 TDD Workflow (Per Feature)

1. **Write a failing test** covering the acceptance criteria
2. **Write the minimum code** to make the test pass
3. **Refactor** while keeping tests green
4. **Commit** with conventional commit message
5. **Push to GitHub** — CI runs test suite

Listing 12.1: Git Workflow Example

```
# Phase 1: Auth feature
git checkout -b feature/auth-register

# TDD cycle
dotnet test tests/TuitionHub.Api.Tests --filter "RegisterGuardian"  # Red
# ... implement RegisterCommandHandler ...
dotnet test tests/TuitionHub.Api.Tests --filter "RegisterGuardian"  # Green
# ... refactor ...

git add .
git commit -m "feat: add guardian registration with email OTP"
git push -u origin feature/auth-register
# → GitHub Actions runs full test suite
# → Create PR → merge to develop
```

Chapter 13

Project Structure

13.1 Complete Directory Layout

Listing 13.1: TuitionHub BD Full Project Tree

```
TuitionHub-BD/
 frontend/                          # Next.js 15 Frontend
    app/
       (marketing)/                # Public landing pages
       (auth)/                     # Login, register, OTP
       (dashboard)/
          guardian/                # Guardian dashboard
          tutor/                   # Tutor dashboard
          admin/                   # Admin panel
       api/                        # BFF API routes
       layout.tsx
       page.tsx
    components/
       ui/                         # shadcn/ui primitives
       magicui/                    # Magic UI components
       forms/                      # Form components
       layout/                     # Navbar, sidebar, footer
    lib/                           # Utilities, API client, Zod schemas
    hooks/                         # Custom React hooks
    stores/                        # Zustand stores

 backend/
    TuitionHub.Domain/             # Entities, Enums, Interfaces
    TuitionHub.Application/        # CQRS, MediatR handlers
    TuitionHub.Infrastructure/     # EF Core, Repositories, JWT, SignalR
    TuitionHub.Api/                # Controllers, Middleware, Program.cs

 ai-service/
    app/
       api/                        # FastAPI endpoints
       core/                       # Matching engine, scoring
       models/                     # Pydantic schemas
       services/                   # Salary predictor, fraud detector
    tests/
    Dockerfile

 database/
```

```
    migrations/                         # EF Core migrations
    seeds/                              # Seed data
    scripts/                            # Utility SQL scripts

docker/
    docker-compose.yml
    Dockerfile.*

docs/                                    # Design docs, API specs
.github/
    workflows/                          # CI/CD pipelines
README.md
```

Chapter 14

Appendix

14.1 Conventional Commit Format

`type(scope): description`

- **feat**: New feature
- **fix**: Bug fix
- **test**: Test additions or changes
- **refactor**: Code restructuring
- **chore**: Build, CI, config changes
- **docs**: Documentation

14.2 Design Principles Applied

- **SOLID**: Single responsibility, Open-closed, Liskov substitution, Interface segregation, Dependency inversion
- **Clean Architecture**: Domain-centric, framework-agnostic core
- **CQRS**: Separated read/write models via MediatR
- **Repository Pattern**: Data access abstraction
- **Result Pattern**: Explicit success/failure, no exceptions for control flow
- **Outbox Pattern**: Reliable async event delivery
- **DRY**: Single source of truth for business rules
- **Pagination**: Cursor-based for large datasets

14.3 Security Considerations

- HTTPS everywhere, HSTS headers
- JWT access tokens: 15 min expiry, in-memory storage
- Refresh tokens: 7 days, HTTP-only Secure cookies, rotation on use
- Password hashing: bcrypt (ASP.NET Identity default)
- Rate limiting: 100 req/min per user (Redis-backed)
- CORS: Whitelisted origins only
- SQL injection: prevented by EF Core parameterized queries
- XSS: React's JSX sanitization + Content Security Policy
- CSRF: Anti-forgery tokens on state-changing endpoints
- File upload: Size limits, MIME validation, virus scanning
- Secrets: Environment variables / Azure Key Vault

14.4 Monitoring & Observability

- **Logging**: Serilog (structured) + Grafana Loki
- **Metrics**: Prometheus — request rate, latency, error rate, DB query perf
- **Tracing**: OpenTelemetry distributed traces across .NET → FastAPI → PostgreSQL
- **Errors**: Sentry for exception aggregation
- **Uptime**: Health check endpoints (./well-known/health)
- **Alerts**: PagerDuty / Slack webhooks on critical failures